

Guía de Laboratorio: Árboles Binarios y BST

Profesor: Brenner Ojeda

Instrucciones generales

En esta guía resolverá problemas clásicos sobre árboles binarios y árboles binarios de búsqueda. Cada ejercicio debe resolverse implementando una función en C++ usando la estructura estándar de un nodo de árbol binario.

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right)
        : val(x), left(left), right(right) {}
};
```

Para cada problema debe entregar:

- Código fuente en C++.
- Explicación breve de la idea principal.
- Análisis de complejidad temporal.
- Análisis de complejidad espacial.

Problema 1: Profundidad máxima de un árbol binario

Dado el nodo raíz de un árbol binario, escriba una función que retorne la profundidad máxima del árbol.

La profundidad máxima se define como el número de nodos en el camino más largo desde la raíz hasta una hoja. Una hoja es un nodo que no tiene hijo izquierdo ni hijo derecho.

Si el árbol está vacío, su profundidad máxima es 0.

Firma sugerida:

```
int maxDepth(TreeNode* root);
```

Ejemplo 1:

Input: root = [3,9,20,null,null,15,7]

Output: 3

Explicación:

El camino más largo puede ser:

3 -> 20 -> 15

o también:

3 -> 20 -> 7

Ambos caminos tienen 3 nodos.

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[0, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$.
- El árbol puede estar vacío.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(h)$ usando recursión, donde h es la altura del árbol.

Prueba tu código AQUÍ.

Problema 2: Árboles iguales

Dados los nodos raíz de dos árboles binarios, determine si ambos árboles son exactamente iguales. Dos árboles binarios son iguales si cumplen las siguientes condiciones:

- Tienen la misma estructura.
- Los nodos correspondientes tienen el mismo valor.

Si ambos árboles están vacíos, se consideran iguales. Si uno está vacío y el otro no, no son iguales.

Firma sugerida:

```
bool isSameTree(TreeNode* p, TreeNode* q);
```

Ejemplo 1:

Input: $p = [1,2,3]$, $q = [1,2,3]$

Output: true

Ejemplo 2:

Input: $p = [1,2]$, $q = [1,null,2]$

Output: false

Explicación:

Aunque ambos árboles contienen los valores 1 y 2, sus estructuras son distintas.

Restricciones de entrada:

- El número de nodos de cada árbol está en el rango $[0, 100]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$.
- Los árboles pueden estar vacíos.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$, donde n es el número mínimo de nodos que se deben comparar.
- Espacio esperado: $O(h)$ por la pila de llamadas recursivas.

Prueba tu código [AQUI](#).

Problema 3: Recorrido por niveles de un árbol binario

Dado el nodo raíz de un árbol binario, retorne el recorrido por niveles de sus nodos.

El recorrido por niveles visita los nodos de arriba hacia abajo y, dentro de cada nivel, de izquierda a derecha.

El resultado debe retornarse como un vector de vectores. Cada vector interno representa los valores de los nodos en un nivel del árbol.

Firma sugerida:

```
vector<vector<int>> levelOrder(TreeNode* root);
```

Ejemplo 1:

Input: root = [3,9,20,null,null,15,7]

Output: [[3],[9,20],[15,7]]

Explicación:

El árbol tiene tres niveles:

Nivel 0: 3

Nivel 1: 9, 20

Nivel 2: 15, 7

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$.
- El árbol puede estar vacío.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(n)$ debido a la cola utilizada para BFS y al vector de respuesta.

Prueba tu código [AQUI](#).

Problema 4: Árbol binario balanceado

Dado el nodo raíz de un árbol binario, determine si el árbol está balanceado en altura.

Un árbol binario está balanceado si, para cada nodo, la diferencia entre la altura de su subárbol izquierdo y la altura de su subárbol derecho es como máximo 1.

La condición debe cumplirse en todos los nodos del árbol, no solo en la raíz.

Firma sugerida:

```
bool isBalanced(TreeNode* root);
```

Ejemplo 1:

Input: root = [3,9,20,null,null,15,7]

Output: true

Ejemplo 2:

Input: root = [1,2,2,3,3,null,null,4,4]

Output: false

Explicación:

En el segundo ejemplo, existe al menos un nodo cuya diferencia de alturas entre sus subárboles es mayor que 1.

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[0, 5000]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$.
- El árbol puede estar vacío.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(h)$ por la recursión.
- Evite soluciones de $O(n^2)$ que recalculen alturas repetidamente.

Prueba tu código [AQUI](#).

Problema 5: Diámetro de un árbol binario

Dado el nodo raíz de un árbol binario, calcule el diámetro del árbol.

El diámetro de un árbol binario se define como el número de aristas en el camino más largo entre dos nodos cualesquiera del árbol. Este camino puede pasar o no por la raíz.

Firma sugerida:

```
int diameterOfBinaryTree(TreeNode* root);
```

Ejemplo 1:

Input: root = [1,2,3,4,5]

Output: 3

Explicación:

El camino más largo puede ser:

4 -> 2 -> 1 -> 3

Este camino tiene 3 aristas.

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[1, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$.
- El árbol no necesariamente está balanceado.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(h)$ por la pila de recursión.
- La solución debe calcular alturas y actualizar el diámetro en una sola exploración.

Prueba tu código [AQUI](#).

Problema 6: Suma de camino

Dado el nodo raíz de un árbol binario y un entero `targetSum`, determine si existe un camino desde la raíz hasta alguna hoja tal que la suma de los valores de los nodos del camino sea igual a `targetSum`.

Un camino válido debe comenzar necesariamente en la raíz y terminar necesariamente en una hoja.

Firma sugerida:

```
bool hasPathSum(TreeNode* root, int targetSum);
```

Ejemplo 1:

```
Input:  root = [5,4,8,11,null,13,4,7,2,null,null,null,1]
        targetSum = 22
```

```
Output: true
```

Explicación:

Existe el camino:

5 -> 4 -> 11 -> 2

La suma es:

5 + 4 + 11 + 2 = 22

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[0, 5000]$.
- $-1000 \leq \text{Node.val} \leq 1000$.
- $-1000 \leq \text{targetSum} \leq 1000$.
- El árbol puede estar vacío.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(h)$ por la recursión.

Prueba tu código [AQUI](#).

Problema 7: Construir árbol desde preorder e inorder

Dados dos arreglos de enteros **preorder** e **inorder**, construya y retorne el árbol binario correspondiente.

El recorrido preorder visita los nodos en el siguiente orden:

raíz -> subárbol izquierdo -> subárbol derecho

El recorrido inorder visita los nodos en el siguiente orden:

subárbol izquierdo -> raíz -> subárbol derecho

Se garantiza que los valores de los nodos son únicos y que ambos arreglos corresponden al mismo árbol binario.

Firma sugerida:

```
TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder);
```

Ejemplo 1:

Input: preorder = [3,9,20,15,7]

inorder = [9,3,15,20,7]

Output: [3,9,20,null,null,15,7]

Explicación:

El primer valor de preorder siempre representa la raíz del árbol. Luego, usando la posición de esa raíz en inorder, se puede separar el subárbol izquierdo del subárbol derecho.

Restricciones de entrada:

- $1 \leq \text{preorder.length} \leq 3000$.
- $\text{inorder.length} = \text{preorder.length}$.
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$.
- Todos los valores son únicos.
- Ambos arreglos representan el mismo árbol binario.

Restricciones de complejidad:

- Tiempo esperado: $O(n)$.
- Espacio esperado: $O(n)$.
- Se recomienda usar una tabla hash para consultar rápidamente la posición de cada valor en inorder.

Prueba tu código [AQUI](#).

Problema 8: Insertar en un árbol binario de búsqueda

Dado el nodo raíz de un árbol binario de búsqueda, también llamado BST, y un valor entero `val`, inserte el nuevo valor en el árbol respetando las propiedades del BST.

Un árbol binario de búsqueda cumple que:

- Todo valor en el subárbol izquierdo de un nodo es menor que el valor del nodo.
- Todo valor en el subárbol derecho de un nodo es mayor que el valor del nodo.

Puede retornar cualquier árbol válido después de la inserción. Se garantiza que `val` no existe previamente en el BST.

Firma sugerida:

```
TreeNode* insertIntoBST(TreeNode* root, int val);
```

Ejemplo 1:

Input: `root = [4,2,7,1,3]`, `val = 5`

Output: `[4,2,7,1,3,5]`

Explicación:

El valor 5 debe insertarse en el subárbol derecho de 4, pero como 5 es menor que 7, debe ubicarse como hijo izquierdo de 7.

Restricciones de entrada:

- El número de nodos del árbol está en el rango $[0, 10^4]$.
- $-10^8 \leq \text{Node.val} \leq 10^8$.
- $-10^8 \leq \text{val} \leq 10^8$.
- Todos los valores del BST son únicos.
- `val` no aparece previamente en el BST.
- El árbol dado cumple la propiedad de BST.

Restricciones de complejidad:

- Tiempo esperado: $O(h)$, donde h es la altura del árbol.
- En el peor caso, si el BST está degenerado, el tiempo puede ser $O(n)$.
- Espacio esperado: $O(h)$ si usa recursión.
- Espacio esperado: $O(1)$ si usa una solución iterativa.

Prueba tu código AQUÍ.